Task-6D

February 8, 2025

0.1 Deakin University

0.2 SIG731- Data Wrangling

Name: Surendra Bahadur Rai

Email: s223940212@deakin.com

Student ID: s223940212

0.3 Task-6D

In this assignment task, we will explore flight-related datasets from three major New York airports: Newark (EWR), John F. Kennedy (JFK), and LaGuardia (LGA) for the year 2013. The datasets contain information on flights, airlines, airports, planes, and weather conditions, providing a rich source for analytical practice.

Our objective is to demonstrate how to load these datasets into Pandas DataFrames and use SQL queries to extract specific insights efficiently. This task will help you understand data loading, querying, and filtering techniques in pandas Python.

0.3.1 Datasets Description

- **Flights:** Information about flight schedules, delays, and destinations.
- **Airlines:** Details of different airline operators.
- **Airports:** Metadata about airport locations and codes.
- **Planes:** Information on airplanes models and their manufacturing details.
- **Weather:** Weather conditions recorded at the airports during flight operations.

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import sqlite3
from pandas.testing import assert_frame_equal
import os

#Python warning ingorning
import warnings
warnings.filterwarnings('ignore')
```

0.4 Reading all nycflights13 dataset (airline, flights, airports, planes, weather)

[2]:
```
#airlines dataset
airlines_df = pd.read_csv("data/nycflights13_airlines.csv.gz",comment="#")
airlines_df.head()
```

[2]:

	carrier	name
0	9E	Endeavor Air Inc.
1	AA	American Airlines Inc.
2	AS	Alaska Airlines Inc.
3	B6	JetBlue Airways
4	DL	Delta Air Lines Inc.

[3]:
```
#flights dataset
flights_df = pd.read_csv("data/nycflights13_flights.csv.gz",comment="#")
flights_df.head()
```

[3]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	\
0	2013	1	1	517.0	515	2.0	830.0	
1	2013	1	1	533.0	529	4.0	850.0	
2	2013	1	1	542.0	540	2.0	923.0	
3	2013	1	1	544.0	545	-1.0	1004.0	
4	2013	1	1	554.0	600	-6.0	812.0	

	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	\
0	819	11.0	UA	1545	N14228	EWR	IAH	227.0	
1	830	20.0	UA	1714	N24211	LGA	IAH	227.0	
2	850	33.0	AA	1141	N619AA	JFK	MIA	160.0	
3	1022	-18.0	B6	725	N804JB	JFK	BQN	183.0	
4	837	-25.0	DL	461	N668DN	LGA	ATL	116.0	

	distance	hour	minute	time_hour
0	1400	5	15	2013-01-01 05:00:00
1	1416	5	29	2013-01-01 05:00:00
2	1089	5	40	2013-01-01 05:00:00
3	1576	5	45	2013-01-01 05:00:00
4	762	6	0	2013-01-01 06:00:00

[4]:
```
#airports dataset
airports_df = pd.read_csv("data/nycflights13_airports.csv.gz",comment="#")
airports_df.head()
```

[4]:

	faa	name	lat	lon	alt	tz	dst	\
0	04G	Lansdowne Airport	41.130472	-80.619583	1044	-5	A	
1	06A	Moton Field Municipal Airport	32.460572	-85.680028	264	-6	A	
2	06C	Schaumburg Regional	41.989341	-88.101243	801	-6	A	
3	06N	Randall Airport	41.431912	-74.391561	523	-5	A	

```
4  09J          Jekyll Island Airport  31.074472 -81.427778    11  -5    A

              tzone
0  America/New_York
1   America/Chicago
2   America/Chicago
3  America/New_York
4  America/New_York
```

[5]:
```
#planes dataset
planes_df = pd.read_csv("data/nycflights13_planes.csv.gz",comment="#")
planes_df.head()
```

[5]:
```
  tailnum    year                     type      manufacturer      model  \
0  N10156  2004.0  Fixed wing multi engine           EMBRAER  EMB-145XR
1  N102UW  1998.0  Fixed wing multi engine  AIRBUS INDUSTRIE   A320-214
2  N103US  1999.0  Fixed wing multi engine  AIRBUS INDUSTRIE   A320-214
3  N104UW  1999.0  Fixed wing multi engine  AIRBUS INDUSTRIE   A320-214
4  N10575  2002.0  Fixed wing multi engine           EMBRAER  EMB-145LR

   engines  seats  speed     engine
0        2     55    NaN  Turbo-fan
1        2    182    NaN  Turbo-fan
2        2    182    NaN  Turbo-fan
3        2    182    NaN  Turbo-fan
4        2     55    NaN  Turbo-fan
```

[6]:
```
#weather dataset
weather_df = pd.read_csv("data/nycflights13_weather.csv.gz",comment="#")
weather_df.head()
```

[6]:
```
  origin  year  month  day  hour   temp   dewp  humid  wind_dir  wind_speed  \
0    EWR  2013      1    1     0  37.04  21.92  53.97     230.0    10.35702
1    EWR  2013      1    1     1  37.04  21.92  53.97     230.0    13.80936
2    EWR  2013      1    1     2  37.94  21.92  52.09     230.0    12.65858
3    EWR  2013      1    1     3  37.94  23.00  54.51     230.0    13.80936
4    EWR  2013      1    1     4  37.94  24.08  57.04     240.0    14.96014

   wind_gust  precip  pressure  visib            time_hour
0  11.918651     0.0    1013.9   10.0  2013-01-01 01:00:00
1  15.891535     0.0    1013.0   10.0  2013-01-01 02:00:00
2  14.567241     0.0    1012.6   10.0  2013-01-01 03:00:00
3  15.891535     0.0    1012.7   10.0  2013-01-01 04:00:00
4  17.215830     0.0    1012.8   10.0  2013-01-01 05:00:00
```

0.4.1 Creating SQLite3 database Flat file

[7]:
```
cwd = os.getcwd()
sqllite3_dbfile = os.path.join(cwd, "sqlite3_datafile.db")
sqllite3_dbfile
```

[7]:
```
'/home/railapy/pyworkspace/deakin/data-wrangling/Task-6D/sqlite3_datafile.db'
```

In current directory folder Create SQLite3 data file “sqlite3_datafile.db”

0.4.2 SQlite3 Database Connection String

[8]:
```
#SQLite3 connection string to flatfile database
conn = sqlite3.connect(sqllite3_dbfile)
```

0.4.3 Pandas Dataframe to SQLite3 Database tables(Airlines_tbl,Flights_tbl,Airports_tbl,Planes_

[9]:
```
airlines_df.to_sql("Airlines_tbl", conn, if_exists='replace',index=False)
flights_df.to_sql("Flights_tbl", conn, if_exists='replace',index=False)
airports_df.to_sql("Airports_tbl", conn,if_exists='replace',index=False)
planes_df.to_sql("Planes_tbl", conn, if_exists='replace',index=False)
weather_df.to_sql("Weather_tbl", conn, if_exists='replace',index=False)
```

[9]:
```
26130
```

Pandas DataFrame Object create SQLite Database Tabless: - airlines_df -> Airlines_tbl - flights_df -> Flights_tbl - airports_df -> Airports_tbl - planes_df - > Planes_tbl - weather_df -> Weather_tbl

using pandas DataFrame.to_sql() method create table

0.5 Q1

[10]:
```
# SQl Query Reading
task1_sql = pd.read_sql_query("""
    SELECT DISTINCT engine FROM Planes_tbl
""", conn)

# Pandas filter engine and drop douplicate and drop the index
task1_pd = planes_df.drop_duplicates(subset=['engine'])[['engine']].
 ↪reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task1_sql,task1_pd)

print("both are equal:",task1_sql.equals(task1_pd))

task1_sql
```

```
both are equal: True
```

[10]:

	engine
0	Turbo-fan
1	Turbo-jet
2	Reciprocating
3	4 Cycle
4	Turbo-shaft
5	Turbo-prop

- Here read_sql_query() method allow you to read sql syntax convert into pandas dataframe Object
- By using Planes_df dataframe , distict engine fild and select the engine filed
- verifying the pandas and SQL Result

0.6 Q2

[11]:
```
#Sql Query Featching
task2_sql = pd.read_sql_query("""
    SELECT DISTINCT type, engine FROM Planes_tbl
""", conn)

# Pandas filter type,engine and drop douplicate only type and drop the index
task2_pd = planes_df.
 ↪drop_duplicates(subset=['type','engine'])[['type','engine']].
 ↪reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task2_sql,task2_pd)

print("both are equal:",task2_sql.equals(task2_pd))
task2_sql
```

```
both are equal: True
```

[11]:

	type	engine
0	Fixed wing multi engine	Turbo-fan
1	Fixed wing multi engine	Turbo-jet
2	Fixed wing single engine	Reciprocating
3	Fixed wing multi engine	Reciprocating
4	Fixed wing single engine	4 Cycle
5	Rotorcraft	Turbo-shaft
6	Fixed wing multi engine	Turbo-prop

- Sql Query Featching Reading pandas DataFrame
- Pandas filter type,engine and drop douplicate only type and drop the index
- test both dataset, pandas and sql query is true or false

0.7 Q3

[12]:
```
#Sql Query Featching Reading pandas DataFrame
task3_sql = pd.read_sql_query("""
    SELECT COUNT(*), engine FROM Planes_tbl GROUP BY engine
""", conn)

# Pandas filter type,engine and count
task3_pd = planes_df.groupby('engine').size().reset_index(name='COUNT(*)')

# Reorder columns to have 'count' first and 'engine' second
task3_pd = task3_pd[['COUNT(*)', 'engine']]

#test both dataset is true or false
assert_frame_equal(task3_sql,task3_pd)

print("both are equal:",task3_sql.equals(task3_pd))
task3_sql
```

```
both are equal: True
```

[12]:

	COUNT(*)	engine
0	2	4 Cycle
1	28	Reciprocating
2	2750	Turbo-fan
3	535	Turbo-jet
4	2	Turbo-prop
5	5	Turbo-shaft

- Sql Query Featching Reading pandas DataFrame
- Pandas filter type,engine and count
- Reorder columns to have 'count' first and 'engine' second
- test both dataset, pandas and sql query is true or false

0.8 Q4

[13]:
```
# SqL Query using
task4_sql = pd.read_sql_query("""
    SELECT COUNT(*), engine, type FROM Planes_tbl GROUP BY engine, type
""", conn)

# 'planes_df' groupby enine, type and count
task4_pd = planes_df.groupby(['engine', 'type']).size().
 ↪reset_index(name='COUNT(*)')

# Reorder columns to match SQL output: count, engine, type
task4_pd = task4_pd[['COUNT(*)', 'engine', 'type']]
```

```
#test both dataset is true or false
assert_frame_equal(task4_sql,task4_pd)

print("both are equal:",task4_sql.equals(task4_pd))

task4_sql
```

```
both are equal: True
```

[13]:

	COUNT(*)	engine	type
0	2	4 Cycle	Fixed wing single engine
1	5	Reciprocating	Fixed wing multi engine
2	23	Reciprocating	Fixed wing single engine
3	2750	Turbo-fan	Fixed wing multi engine
4	535	Turbo-jet	Fixed wing multi engine
5	2	Turbo-prop	Fixed wing multi engine
6	5	Turbo-shaft	Rotorcraft

- SqL Query using create pandas object
- 'planes_df' groupby enine, type and count
- Reorder columns to match SQL output: count, engine, type
- test both dataset, pandas and sql query is true or false

0.9 Q5

[14]:
```
#SQL Query
task5_sql = pd.read_sql_query("""
SELECT MIN(year), AVG(year), MAX(year), engine, manufacturer FROM Planes_tbl␣
 ↪GROUP BY engine, manufacturer
""", conn)

# pandas gropup by syntax apply aggregate function min, max, mean
task5_pd = planes_df.groupby(['engine', 'manufacturer'])['year'].agg(
    min_year=('min'),
    avg_year=('mean'),
    max_year=('max')
).reset_index()

# Rename columns to match SQL
task5_pd = task5_pd.rename(columns={
    'min_year': 'MIN(year)',
    'avg_year': 'AVG(year)',
    'max_year': 'MAX(year)'
})
# Reorder columns to match SQL output: count, engine, type
task5_pd = task5_pd[['MIN(year)', 'AVG(year)',␣
 ↪'MAX(year)','engine','manufacturer']]
```

```
#test both dataset is true or false
assert_frame_equal(task5_sql,task5_pd)

print("both are equal:",task5_sql.equals(task5_pd))

task5_sql.head(10)
```

```
both are equal: True
```

[14]:

	MIN(year)	AVG(year)	MAX(year)	engine	manufacturer
0	1975.0	1975.000000	1975.0	4 Cycle	CESSNA
1	NaN	NaN	NaN	4 Cycle	JOHN G HESS
2	NaN	NaN	NaN	Reciprocating	AMERICAN AIRCRAFT INC
3	2007.0	2007.000000	2007.0	Reciprocating	AVIAT AIRCRAFT INC
4	NaN	NaN	NaN	Reciprocating	BARKER JACK L
5	1959.0	1971.142857	1983.0	Reciprocating	CESSNA
6	2007.0	2007.000000	2007.0	Reciprocating	CIRRUS DESIGN CORP
7	1959.0	1959.000000	1959.0	Reciprocating	DEHAVILLAND
8	1956.0	1956.000000	1956.0	Reciprocating	DOUGLAS
9	2007.0	2007.000000	2007.0	Reciprocating	FRIEDEMANN JON

- Appy Sql Query Featching to pandas DataFrame
- Pandas gropup by (engine, manufacture) and appply agreegate function mean, max and mean
- Rename columns to match SQL query output
- Reorder columns to match SQL output: count, engine, type
- test both dataset, pandas and sql query is true or false

0.10 Q6

[15]:
```
# sql query
task6_sql = pd.read_sql_query("""
    SELECT * FROM Planes_tbl WHERE speed IS NOT NULL
    """, conn)

# define task6_pd and filter the Not null value of speed from Pandas DataFrame
task6_pd = planes_df[planes_df['speed'].notna()].reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task6_sql,task6_pd)

print("both are equal:",task6_sql.equals(task6_pd))
task6_sql.head(10)
```

```
both are equal: True
```

[15]:

	tailnum	year	type	manufacturer	model	engines	seats	speed	engine
0	N201AA	1959.0	Fixed wing single engine	CESSNA	150	1	2	90.0	Reciprocating
1	N202AA	1980.0	Fixed wing multi engine	CESSNA	421C	2	8	90.0	Reciprocating
2	N350AA	1980.0	Fixed wing multi engine	PIPER	PA-31-350	2	8	162.0	Reciprocating
3	N364AA	1973.0	Fixed wing multi engine	CESSNA	310Q	2	6	167.0	Reciprocating
4	N378AA	1963.0	Fixed wing single engine	CESSNA	172E	1	4	105.0	Reciprocating
5	N381AA	1956.0	Fixed wing multi engine	DOUGLAS	DC-7BF	4	102	232.0	Reciprocating
6	N425AA	1968.0	Fixed wing single engine	PIPER	PA-28-180	1	4	107.0	Reciprocating
7	N508AA	1975.0	Rotorcraft	BELL	206B	1	5	112.0	Turbo-shaft
8	N519MQ	1983.0	Fixed wing single engine	CESSNA	A185F	1	6	127.0	Reciprocating
9	N525AA	1980.0	Fixed wing multi engine	PIPER	PA-31-350	2	8	162.0	Reciprocating

- Appy Sql Query Featching to pandas DataFrame
- define task6_pd and filter the Not null value of speed from Pandas DataFrame
- test both dataset, pandas and sql query is true or false

0.11 Q7

[16]:
```
#sql Query
task7_sql = pd.read_sql_query("""SELECT tailnum FROM Planes_tbl WHERE seats␣
 ↪BETWEEN 150 AND 210 AND year >= 2011""", conn)

# Pandas Query
task7_pd = planes_df[
    (planes_df['seats'].between(150, 210)) &
    (planes_df['year'] >= 2011)
][['tailnum']].reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task7_sql,task7_pd)

print("both are equal:",task7_sql.equals(task7_pd))
task7_sql
```

```
both are equal: True
```

[16]:
```
   tailnum
0   N150UW
1   N151UW
2   N152UW
3   N153UW
4   N154UW
..     …
87  N851VA
88  N852VA
89  N853VA
90  N854VA
91  N855VA

[92 rows x 1 columns]
```

- Appy Sql Query Featching to pandas DataFrame
- define task7_pd and apply between filter 150 to 210 seats, then
- greater then equal 2011 years only the tailnum filter.
- test both dataset, pandas and sql query is true or false

0.12 Q8

[17]:
```
#SQL Query
task8_sql = pd.read_sql_query("""SELECT tailnum, manufacturer, seats FROM␣
 ↪Planes_tbl
WHERE manufacturer IN ("BOEING", "AIRBUS", "EMBRAER") AND seats>390""", conn)

#Pandas filter
task8_pd = planes_df[
    (planes_df['manufacturer'].isin(["BOEING", "AIRBUS", "EMBRAER"])) &
    (planes_df['seats'] > 390)
][['tailnum', 'manufacturer', 'seats']].reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task8_sql,task8_pd)

print("both are equal:",task8_sql.equals(task8_pd))
task8_sql.head(10)
```

```
both are equal: True
```

[17]:

	tailnum	manufacturer	seats
0	N206UA	BOEING	400
1	N228UA	BOEING	400
2	N272AT	BOEING	400
3	N57016	BOEING	400
4	N670US	BOEING	450

```
5  N77012       BOEING    400
6  N777UA       BOEING    400
7  N78003       BOEING    400
8  N78013       BOEING    400
9  N787UA       BOEING    400
```

- Appy Sql Query Featching to pandas DataFrame
- define task8_pd and filter manufacturer is “BOEING”, “AIRBUS”, “EMBRAER”]
- greater then equal 390 seats of planes.
- test both dataset, pandas and sql query is true or false

0.13 Q9

[18]:
```
#SQL Query Reading pandas sql

task9_sql = pd.read_sql_query("""SELECT DISTINCT year, seats FROM Planes_tbl
WHERE year >= 2012 ORDER BY year ASC, seats DESC""", conn)

# Replicate Similar SQL query to pandas
task9_df = (
    planes_df.loc[:, ['year', 'seats']]  # Select only 'year' and 'seats'␣
 ↪columns
    .query('year >= 2012')  # Filter rows where year >= 2012
    .drop_duplicates()  # Get distinct combinations
    .sort_values(by=['year', 'seats'], ascending=[True, False]) # Sort
    .reset_index(drop=True)
)

#test both dataset is true or false
assert_frame_equal(task9_sql,task9_df)

print("both are equal:",task9_sql.equals(task9_df))
task9_sql.head(10)
```

```
both are equal: True
```

[18]:

	year	seats
0	2012.0	379
1	2012.0	377
2	2012.0	260
3	2012.0	222
4	2012.0	200
5	2012.0	191
6	2012.0	182
7	2012.0	149
8	2012.0	140
9	2012.0	20

- Appy Sql Query Featching to pandas DataFrame & define task9_sql
- define task9_df and Select only 'year' and 'seats' columns
- Filter rows where year >= 2012
- distinct combinations
- sorted years and seats and apply year accending and seats decending
- test both dataset, pandas and sql query is true or false

0.14 Q10

[19]:
```
#SqL query
task10_sql = pd.read_sql_query("""SELECT DISTINCT year, seats FROM Planes_tbl
WHERE year >= 2012 ORDER BY seats DESC, year ASC""", conn)

# Replicate the SQL query in pandas
task10_df = (
    planes_df.loc[planes_df['year'] >= 2012, ['year', 'seats']]  # Filter and␣
 ↪select columns
    .drop_duplicates()  # Get distinct only
    .sort_values(by=['seats', 'year'], ascending=[False, True])  # Sort by␣
 ↪seats DESC, year ASC
    .reset_index(drop=True)
)
#test both dataset is true or false
assert_frame_equal(task10_sql,task10_df)
print("both are equal:",task10_sql.equals(task10_df))
task10_sql
```

```
both are equal: True
```

[19]:

	year	seats
0	2012.0	379
1	2013.0	379
2	2012.0	377
3	2013.0	377
4	2012.0	260
5	2012.0	222
6	2013.0	222
7	2012.0	200
8	2013.0	200
9	2013.0	199
10	2012.0	191
11	2013.0	191
12	2012.0	182
13	2013.0	182
14	2012.0	149
15	2012.0	140
16	2013.0	140

```
17  2013.0     95
18  2012.0     20
19  2013.0     20
20  2012.0      5
```

- Appy Sql Query Featching to pandas DataFrame & define task10_sql
- define task10_df and Filter and select columns years and seats where years greater then 2012
- removing doublicates only distic values
- ort by seats DESC, year ASC
- drop the index
- test both dataset, pandas and sql query is true or false

0.15 Q11

[20]:
```
# SQL Query
task11_sql = pd.read_sql_query("""SELECT manufacturer, COUNT(*) FROM Planes_tbl
WHERE seats > 200 GROUP BY manufacturer""", conn)

# Replicate the SQL query in pandas
task11_pd = (
    planes_df[planes_df['seats'] > 200]  # Filter rows where seats > 200
    .groupby('manufacturer', as_index=False)  # Group by 'manufacturer'
    .size()  # Count rows in each group
    .rename(columns={'size': 'COUNT(*)'})  # Rename the count column
    .reset_index(drop=True)
)

#test both dataset is true or false
assert_frame_equal(task11_sql,task11_pd)

print("both are equal:",task11_sql.equals(task11_pd))

task11_sql
```

```
both are equal: True
```

[20]:

	manufacturer	COUNT(*)
0	AIRBUS	66
1	AIRBUS INDUSTRIE	4
2	BOEING	225

- Appy Sql Query Featching to pandas DataFrame & define task11_sql
- define task11_pd and Filter seats where greater then 200
- group by manufacturer
- count rows of each query
- Rename the columns count size same as SQL output
- test both dataset, pandas and sql query is true or false

0.16 Q12

[21]:
```
# SQL Query featching
task12_sql = pd.read_sql_query("""SELECT manufacturer, COUNT(*) FROM Planes_tbl
GROUP BY manufacturer HAVING COUNT(*) > 10""", conn)

# Replicate the SQL query in pandas
task12_pd = (
     planes_df.groupby('manufacturer')
    .size()
    .reset_index(name='count')
    .query('count > 10')[['manufacturer','count']]
    .rename(columns={'count': 'COUNT(*)'})
    .reset_index(drop=True)
)

#test both dataset is true or false
assert_frame_equal(task12_sql,task12_pd)

print("both are equal:",task12_sql.equals(task12_pd))

task12_pd
```

```
both are equal: True
```

[21]:

	manufacturer	COUNT(*)
0	AIRBUS	336
1	AIRBUS INDUSTRIE	400
2	BOEING	1630
3	BOMBARDIER INC	368
4	EMBRAER	299
5	MCDONNELL DOUGLAS	120
6	MCDONNELL DOUGLAS AIRCRAFT CO	103
7	MCDONNELL DOUGLAS CORPORATION	14

- Appy Sql Query Featching to pandas DataFrame & define task12_sql
- define task12_pd and group bye manufacturer
- count rows of each query
- Having cout have greater then 10 and only show the manufacturer and count
- renaming the count to to count(*) smilary output of sql query
- test both dataset, pandas and sql query is true or false

0.17 Q13

[22]:
```
# SQL Reading panda dataframe
task13_sql = pd.read_sql_query("""SELECT manufacturer, COUNT(*) FROM Planes_tbl
WHERE seats > 200 GROUP BY manufacturer HAVING COUNT(*) > 10""", conn)
```

```
# Replicate the SQL query in pandas ways
task13_pd = (
    planes_df[planes_df['seats'] > 200]  # WHERE seats > 200
    .groupby('manufacturer')
    .size()  # COUNT(*)
    .reset_index(name='count')
    .query('count > 10')  # HAVING COUNT(*) > 10
    .rename(columns={'count': 'COUNT(*)'}) #Rename the count column similar␣
 ↪orther of sql output
    .reset_index(drop=True)
)

#test both dataset is true or false
assert_frame_equal(task13_sql,task13_pd)

print("both are equal:",task13_sql.equals(task13_pd))

task13_sql
```

```
both are equal: True
```

[22]:

	manufacturer	COUNT(*)
0	AIRBUS	66
1	BOEING	225

- Appy Sql Query Featching to pandas DataFrame & define task13_sql
- define task13_pd and filter the seats where greater then 200
- group by manufacture and count each size
- Having cout have greater then 10 and only show the manufacturer and count
- renaming the count to to count(*) smilary output of sql query
- test both dataset, pandas and sql query is true or false

0.18 Q14

[23]:
```
# SQL reading
task14_sql= pd.read_sql_query("""SELECT manufacturer, COUNT(*) AS howmany
FROM Planes_tbl
GROUP BY manufacturer
ORDER BY howmany DESC LIMIT 10""", conn)

# converting to pandas syntax
task14_pd = (
    planes_df.groupby('manufacturer')
    .size()
    .reset_index(name='howmany')
    .sort_values(by='howmany', ascending=False)  # ORDER BY howmany DESC
    .head(10)  # LIMIT 10
```

```
    .reset_index(drop=True)
)

#test both dataset is true or false
assert_frame_equal(task14_sql,task14_pd)

print("both are equal:",task14_sql.equals(task14_pd))

task14_sql
```

```
both are equal: True
```

[23]:

	manufacturer	howmany
0	BOEING	1630
1	AIRBUS INDUSTRIE	400
2	BOMBARDIER INC	368
3	AIRBUS	336
4	EMBRAER	299
5	MCDONNELL DOUGLAS	120
6	MCDONNELL DOUGLAS AIRCRAFT CO	103
7	MCDONNELL DOUGLAS CORPORATION	14
8	CESSNA	9
9	CANADAIR	9

- Appy Sql Query Featching to pandas DataFrame & define task14_sql
- define task14_pd and group by manufacture and count each of size
- order by howmany and sorted decending
- limit 10
- test both dataset, pandas and sql query is true or false

0.19 Q15

[24]:
```
#SQL featching query
task15_sql = pd.read_sql_query("""SELECT Flights_tbl.*, Planes_tbl.year AS␣
 ↪plane_year, Planes_tbl.speed AS plane_speed,
Planes_tbl.seats AS plane_seats
FROM Flights_tbl LEFT JOIN Planes_tbl ON Flights_tbl.tailnum=Planes_tbl.
 ↪tailnum""", conn)

#Pandas converstion of sql
task15_pd = (
            flights_df.merge(planes_df[['tailnum', 'year', 'speed', 'seats']],␣
 ↪on='tailnum', how='left')
            .rename(columns={'year_x': 'year',  'year_y': 'plane_year', ␣
 ↪'speed': 'plane_speed', 'seats': 'plane_seats'})
            .reset_index(drop=True)
            )
```

```
#test both dataset is true or false
assert_frame_equal(task15_sql,task15_pd)

print("both are equal:",task15_sql.equals(task15_pd))

task15_sql.head()
```

```
both are equal: True
```

[24]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	\
0	2013	1	1	517.0	515	2.0	830.0	
1	2013	1	1	533.0	529	4.0	850.0	
2	2013	1	1	542.0	540	2.0	923.0	
3	2013	1	1	544.0	545	-1.0	1004.0	
4	2013	1	1	554.0	600	-6.0	812.0	

	sched_arr_time	arr_delay	carrier	…	origin	dest	air_time	distance	\
0	819	11.0	UA	…	EWR	IAH	227.0	1400	
1	830	20.0	UA	…	LGA	IAH	227.0	1416	
2	850	33.0	AA	…	JFK	MIA	160.0	1089	
3	1022	-18.0	B6	…	JFK	BQN	183.0	1576	
4	837	-25.0	DL	…	LGA	ATL	116.0	762	

	hour	minute	time_hour	plane_year	plane_speed	plane_seats
0	5	15	2013-01-01 05:00:00	1999.0	NaN	149.0
1	5	29	2013-01-01 05:00:00	1998.0	NaN	149.0
2	5	40	2013-01-01 05:00:00	1990.0	NaN	178.0
3	5	45	2013-01-01 05:00:00	2012.0	NaN	200.0
4	6	0	2013-01-01 06:00:00	1991.0	NaN	178.0

```
[5 rows x 22 columns]
```

- Appy Sql Query Featching to pandas DataFrame & define task15_sql
- define task15_pd and merge the flight dataset and plance dataset where filter plane dataset 'tailnum', 'year', 'speed', 'seats' and joined on tailumnu left
- renameing some field as per sql output
- dropping the index values
- test both dataset, pandas and sql query is true or false

0.20 Q16

[25]:
```
#Feaching SQL query
task16_sql = pd.read_sql_query("""SELECT Planes_tbl.*, Airlines_tbl.* FROM
(SELECT DISTINCT carrier, tailnum FROM Flights_tbl) AS cartail
INNER JOIN Planes_tbl ON cartail.tailnum=Planes_tbl.tailnum
INNER JOIN Airlines_tbl ON cartail.carrier=Airlines_tbl.carrier""", conn)
```

```
# Select distinct carrier and tailnum from flights_df
cartail = flights_df[['carrier', 'tailnum']].drop_duplicates()

# Perform inner joins with planes_df and airlines_df
task16_pd = (
    cartail.merge(planes_df, on='tailnum', how='inner')
           .merge(airlines_df, on='carrier', how='inner')
)

# Rearrange columns to match SQL order and reset the index
task16_pd = task16_pd[planes_df.columns.tolist() + airlines_df.columns.
 ↪tolist()].reset_index(drop=True)

#data types match between SQL and pandas DataFrame
task16_pd = task16_pd.astype(task16_sql.dtypes.to_dict())

# Sort both DataFrames by all columns for proper comparison
task16_sql = task16_sql.sort_values(by=task16_sql.columns.tolist()).
 ↪reset_index(drop=True)
task16_pd = task16_pd.sort_values(by=task16_pd.columns.tolist()).
 ↪reset_index(drop=True)

#test both dataset is true or false
assert_frame_equal(task16_sql,task16_pd)
print("both are equal:",task16_sql.equals(task16_pd))

task16_sql
```

```
both are equal: True
```

[25]:

	tailnum	year	type	manufacturer	\
0	N10156	2004.0	Fixed wing multi engine	EMBRAER	
1	N102UW	1998.0	Fixed wing multi engine	AIRBUS INDUSTRIE	
2	N103US	1999.0	Fixed wing multi engine	AIRBUS INDUSTRIE	
3	N104UW	1999.0	Fixed wing multi engine	AIRBUS INDUSTRIE	
4	N10575	2002.0	Fixed wing multi engine	EMBRAER	
…	…	…	…	…	
3334	N997AT	2002.0	Fixed wing multi engine	BOEING	
3335	N997DL	1992.0	Fixed wing multi engine	MCDONNELL DOUGLAS AIRCRAFT CO	
3336	N998AT	2002.0	Fixed wing multi engine	BOEING	
3337	N998DL	1992.0	Fixed wing multi engine	MCDONNELL DOUGLAS CORPORATION	
3338	N999DN	1992.0	Fixed wing multi engine	MCDONNELL DOUGLAS CORPORATION	

	model	engines	seats	speed	engine	carrier	\
0	EMB-145XR	2	55	NaN	Turbo-fan	EV	
1	A320-214	2	182	NaN	Turbo-fan	US	

```
2      A320-214        2    182    NaN  Turbo-fan      US
3      A320-214        2    182    NaN  Turbo-fan      US
4     EMB-145LR        2     55    NaN  Turbo-fan      EV
…           …      …     …     …        …     …
3334    717-200        2    100    NaN  Turbo-fan      FL
3335      MD-88        2    142    NaN  Turbo-fan      DL
3336    717-200        2    100    NaN  Turbo-fan      FL
3337      MD-88        2    142    NaN  Turbo-jet      DL
3338      MD-88        2    142    NaN  Turbo-jet      DL

                            name
0         ExpressJet Airlines Inc.
1                  US Airways Inc.
2                  US Airways Inc.
3                  US Airways Inc.
4         ExpressJet Airlines Inc.
…                              …
3334  AirTran Airways Corporation
3335         Delta Air Lines Inc.
3336  AirTran Airways Corporation
3337         Delta Air Lines Inc.
3338         Delta Air Lines Inc.

[3339 rows x 11 columns]
```

- Appy Sql Query Featching to pandas DataFrame & define task16_sql
- define task16_pd and elect distinct carrier and tailnum from flights_df
- Perform inner joins with planes_df and airlines_df
- Rearrange columns to match SQL order and reset the index
- data types match between SQL and pandas DataFrame
- Sort both DataFrames by all columns for proper comparison
- test both dataset, pandas and sql query is true or false

0.21 Q17

[26]:
```
#SQL Query feaching

task17_sql = pd.read_sql_query("""
SELECT flights2.*, atemp, ahumid
FROM (
SELECT * FROM Flights_tbl WHERE origin='EWR'
) AS flights2 LEFT JOIN (
SELECT year, month, day, AVG(temp) AS atemp,
AVG(humid) AS ahumid
FROM Weather_tbl
WHERE origin='EWR'
GROUP BY year, month, day ) AS weather2
```

```
ON flights2.year=weather2.year
AND flights2.month=weather2.month
AND flights2.day=weather2.day""", conn)

# DataFrames weather_df from SQL and Pandas
weather_agg = (
    weather_df.query("origin == 'EWR'")
    .groupby(['year', 'month', 'day'], as_index=False)
    .agg(atemp=('temp', 'mean'), ahumid=('humid', 'mean'))
)

task17_pd = (
    flights_df.query("origin == 'EWR'")
    .merge(weather_agg, on=['year', 'month', 'day'], how='left')
    .reindex(columns=list(flights_df.columns) + ['atemp', 'ahumid'])
)

# Align column orders and reset indices before comparison
task17_sql = task17_sql.sort_index(axis=1).reset_index(drop=True)

task17_pd = task17_pd.sort_index(axis=1).reset_index(drop=True)

#test both dataset is true or false
try:
    assert_frame_equal(task17_sql, task17_pd)
    print("Both are equal: True")
except AssertionError:
    print("Both are equal: False")

task17_sql
```

```
Both are equal: True
```

[26]:

	ahumid	air_time	arr_delay	arr_time	atemp	carrier	day	\
0	58.386087	227.0	11.0	830.0	38.4800	UA	1	
1	58.386087	150.0	12.0	740.0	38.4800	UA	1	
2	58.386087	158.0	19.0	913.0	38.4800	B6	1	
3	58.386087	361.0	-14.0	923.0	38.4800	UA	1	
4	58.386087	337.0	-8.0	854.0	38.4800	UA	1	
…	…	…	…	…	…	…	…	
120830	69.806250	47.0	11.0	2250.0	62.9075	EV	30	
120831	69.806250	37.0	-23.0	2245.0	62.9075	UA	30	
120832	69.806250	39.0	-16.0	2250.0	62.9075	EV	30	
120833	69.806250	120.0	57.0	2339.0	62.9075	EV	30	
120834	69.806250	318.0	42.0	112.0	62.9075	UA	30	

	dep_delay	dep_time	dest	…	flight	hour	minute	month	origin	\

```
0             2.0     517.0  IAH  …    1545     5      15      1     EWR
1            -4.0     554.0  ORD  …    1696     5      58      1     EWR
2            -5.0     555.0  FLL  …     507     6       0      1     EWR
3            -2.0     558.0  SFO  …    1124     6       0      1     EWR
4            -1.0     559.0  LAS  …    1187     6       0      1     EWR
…             …        …  …  …    …   …     …    …      …
120830       13.0    2142.0  PWM  …    4509    21      29      9     EWR
120831       -7.0    2149.0  BOS  …     523    21      56      9     EWR
120832       -9.0    2150.0  MHT  …    3842    21      59      9     EWR
120833       72.0    2211.0  STL  …    4672    20      59      9     EWR
120834       80.0    2233.0  SFO  …     471    21      13      9     EWR

       sched_arr_time  sched_dep_time  tailnum            time_hour  year
0                 819             515   N14228  2013-01-01 05:00:00  2013
1                 728             558   N39463  2013-01-01 05:00:00  2013
2                 854             600   N516JB  2013-01-01 06:00:00  2013
3                 937             600   N53441  2013-01-01 06:00:00  2013
4                 902             600   N76515  2013-01-01 06:00:00  2013
…                  …               …      …                   …   …
120830           2239            2129   N12957  2013-09-30 21:00:00  2013
120831           2308            2156   N813UA  2013-09-30 21:00:00  2013
120832           2306            2159   N10575  2013-09-30 21:00:00  2013
120833           2242            2059   N12145  2013-09-30 20:00:00  2013
120834             30            2113   N578UA  2013-09-30 21:00:00  2013

[120835 rows x 21 columns]
```

- Appy Sql Query Featching to pandas DataFrame & define task17_sql
- define weather_agg object where flter the 'EWR' apply the agrregate function
- define task17_pd and merge the flights_df dataset and weather_agg where aplly the left joined
- Align column orders and reset indices before comparison
- test both dataset, pandas and sql query is true or false

[ ]: